

프로그램 명칭 : 인물대백과사 자서전 전자책 편집기(웹)

버전 : 0.9 (베타)

작성 언어 : JavaScript, HTML/CSS, TypeScript(Deno), SQL(PostgreSQL)

실행 환경 : 웹 브라우저 + Supabase(데이터베이스 · 인증 · 파일 저장 · 서버 함수)

발체 작성일 : 2026. 10. 01.

발체 기준 : 이 프로그램을 위해 작성한 소스코드 중 특징을 이루는 부분을 순서대로 발체함.

외부 공개 라이브러리(supabase-js, qrcode-generator)는 제외함.

각 줄 앞의 숫자는 원본 파일의 줄 번호, : 는 중략.

␣, 는 코드가 아님: 원본의 한 줄이 지면 폭(100칸)을 넘어 다음 줄로 이어 적었다는 표시.

보안을 위해 접속 키 값은 <공개 키 생략>으로 가림.

■ 1. 페이지 모델: 신국판 152×225mm, 여백·글상자·미디어 페이지 틀

파일: studio/index.html

```
321|const PW = 152, PH = 225, PT_CQW = (4/3)/(PW*3.7795275591)*100; // 1pt = 얼마의 cqw
322|const LIMIT = { image:15, video:50, audio:30 }; // MB
323|const MX = 16, MT = 18, MB = 18, CW = PW - MX*2; // 여백(mm): 좌우 16 · 위아래 18 → 본문 폭
␣,120mm
324|const LIM = { x0:MX, x1:PW-MX, y0:MT, y1:PH-MB };
: (중략)
341|const T = (x,y,w,h,html,o={}) => ({id:uid(),type:'text',x,y,w,h,html,fs:12,align:'left',color:
␣, '#1B2B25',font:'serif',b:false,i:false,...o});
: (중략)
666|const blankPage = () => ({id:uid(),name:`새 페이지 ${state.pages.length+1}`,bg:'#FFFFFF',els:[
␣,T(MX,MT,CW,PH-MT-MB, '')]]});
667|// 사진·영상·노래 페이지 틀: 제목 · 콘텐츠(자리) · 설명
668|const TITLE = ph => T(MX,24,CW,12, '', {fs:18,b:true,color:'#0B4536',ph});
669|const DESC = (y,h,ph,o={}) => T(MX,y,CW,h, '', {fs:11,align:'center',color:'#3D4B45',ph,...o});
670|function mediaPageTemplate(kind, name){
671|  const slot=(y,h)=>({id:uid(),type:'slot',kind,x:MX,y,w:CW,h});
672|  if(kind==='image') return {id:uid(),name:name||'사진 페이지',bg:'#FFFFFF',els:[TITLE('사진
␣,제목 (예: 그해 여름의 바다)'),slot(44,110),DESC(160,40,'사진 설명을 적어 보세요\n(예: 2026년
␣,여름, 동해. 파도 소리가 오래 남았다.)')]];
673|  if(kind==='video') return {id:uid(),name:name||'영상 페이지',bg:'#FFFFFF',els:[TITLE('영상
␣,제목 (예: 그날의 바다)'),slot(48,CW*9/16),DESC(48+CW*9/16+8,40,'영상 설명을 적어 보세요\n(종이
␣,로 인쇄하면 이 자리에 QR 코드가 들어가요)')]];
674|  return {id:uid(),name:name||'노래 페이지',bg:'#FBF1EA',els:[TITLE('노래 제목 (예: 너라는 등
␣,대)'),slot(44,34),DESC(86,PH-MB-86,'노래에 담긴 이야기나 가사를 적어 보세요\n\n(예)\n어두운 바
␣,다 위 작은 불빛 하나\n너라는 등대가 나를 불렀지')]];
675|}
676|function insertPageAt(i, kind='text'){
677|  endEdit(); snap(); const p = kind==='text' ? blankPage() : mediaPageTemplate(kind);
678|  state.pages.splice(i,0,p); state.cur=i;
679|  if(kind==='text'){ state.sel=p.els[0].id; renderAll(); startEdit(p.els[0].id); }
680|  else { state.sel=null; renderAll(); toast(`줄무늬 자리를 눌러 ${kind==='image'? '사진': kind==='
␣,='video'? '영상': '노래'}을(를) 넣으세요`); }
681|}
```

■ 2. 글 흐름 엔진: 넘친 글은 다음 쪽으로, 빈자리가 생기면 끌어오기

파일: studio/index.html

```
702|/* ===== 워드처럼 넘친 글을 다음 페이지로 ===== */
703|function textPositions(root){ const out=[]; const w=document.createTreeWalker(root,
  ↳NodeFilter.SHOW_TEXT); let n; while((n=w.nextNode())) for(let k=0;k<n.length;k++) out.push([n,
  ↳k]); return out; }
704|function flowOverflow(e, quiet=false){
705|  const n=elNode(e.id); if(!n) return false; const tx=n.querySelector('.tx');
706|  if(tx.scrollHeight<=tx.clientHeight+2 || e.y+e.h < LIM.y1-0.05) return false;  // 아래 여백
  ↳까지 자란 뒤에도 넘칠 때만
707|  tx.scrollTop=0; // 커서를 따라 상자 속이 스크롤되어 있으면 글자 위치가 어긋나므로 맨 위로 되
  ↳돌린 뒤 잰다
708|  const bottom=tx.getBoundingClientRect().bottom-2, pos=textPositions(tx); if(pos.length<2)
  ↳return false;
709|  // 글자 하나의 아래끝이 상자 안에 있는지. 줄 끝 공백처럼 크기가 0인 글자는 가까운 앞 글자로
  ↳판단(그래야 긴 글을 붙여 넣어도 정확히 자른다)
710|  const fits=i=>{ for(let j=i;j>=Math.max(0,i-8);j--){ const r=document.createRange();
  ↳r.setStart(pos[j][0],pos[j][1]); r.setEnd(pos[j][0],pos[j][1]+1); const rects=
  ↳r.getClientRects(); const b=rects.length?rects[rects.length-1]:null; if(b&&b.height>0) return
  ↳b.bottom<=bottom; } return true; };
711|  let lo=0, hi=pos.length-1; if(!fits(0)) return false;
712|  while(lo<hi){ const mid=Math.ceil((lo+hi)/2); if(fits(mid)) lo=mid; else hi=mid-1; }
713|  let cut=lo+1; if(cut>=pos.length) return false;
714|  for(let k=cut;k>Math.max(0,cut-24);k--){ if(/\\s/.test(pos[k-1][0].data[pos[k-1][1]])){ cut=
  ↳k; break; } } // 단어 중간에서 자르지 않기
715|  // 커서가 잘리는 뒷부분에 있었는지(그러면 다음 페이지에서 이어 쓴다)
716|  const s=getSelection(); let caretAfter=false, caretOffsetInMoved=0;
717|  const cutR=document.createRange(); cutR.setStart(pos[cut][0],pos[cut][1]); cutR.setEndAfter(
  ↳tx.lastChild);
718|  if(s.rangeCount&&tx.contains(s.anchorNode)){ const c=document.createRange(); c.setStart(pos[
  ↳cut][0],pos[cut][1]); c.setEnd(s.anchorNode,s.anchorOffset);
719|    caretAfter = cutR.isPointInRange(s.anchorNode,s.anchorOffset); if(caretAfter)
  ↳caretOffsetInMoved=c.toString().length; }
720|  snap();
721|  const holder=document.createElement('div'); holder.appendChild(cutR.extractContents());
722|  const moved=sanitizeText(holder.innerHTML).replace(/^(\\s|&nbsp;)+/, ''); e.html=sanitizeText(
  ↳tx.innerHTML); tx.innerHTML=e.html;
723|  // 다음 페이지가 이어 쓰기 페이지면 앞에 붙이고, 아니면 새 페이지를 끼워 넣는다
724|  const next=state.pages[state.cur+1]; let target=next&&next.flowFrom===page().id ?
  ↳next.els.find(x=>x.type==='text') : null;
725|  if(target) target.html = moved + (plain(target.html)?' '+target.html: '');
726|  else { const np=blankPage(); np.name=page().name+' (이어서)'; np.flowFrom=page().id; target=
  ↳np.els[0];
727|    Object.assign(target,{html:moved,fs:e.fs,font:e.font,color:e.color,align:e.align,b:e.b,i:
  ↳e.i,u:e.u}); state.pages.splice(state.cur+1,0,np); }
728|  if(caretAfter){ state.editing=false; state.cur++; state.sel=target.id; renderAll();
```

```

    729|   ㄴ, startEdit(target.id); placeCaretAt(ㄴnode(target.id).querySelector('.tx'), Math.max(0,
    ㄴ, caretOffsetInMoved - holder.textContent.match(/\s*/)[0].length)); }
730|   else { renderThumbs(); autoGrow(e); }
731|   if(!quiet) toast('글이 넘쳐서 다음 페이지로 이어졌어요');
732|   const t2=selEl(); if(t2&&t2.id===target.id) requestAnimationFrame(=>{ autoGrow(t2);
    ㄴ, flowOverflow(t2); }); // 다음 페이지도 넘치면 계속
733|   return true;
734| }
735| // 워드처럼: 앞 페이지 글을 지워 자리가 생기면 다음 「(이어서)」 페이지의 글을 끌어 온다
736| function caretOffset(tx){ const s=getSelection(); if(!s.rangeCount||!tx.contains(
    ㄴ, s.anchorNode)) return null; const r=document.createRange(); r.selectNodeContents(tx);
    ㄴ, r.setEnd(s.anchorNode, s.anchorOffset); return r.toString().length; }
737| function pullBack(e){
738|   const pi=state.cur, me=page(), next=state.pages[pi+1];
739|   if(!next || next.flowFrom!==me.id || e.type!=='text' || e.y+e.h < LIM.y1-0.05) return false;
740|   const n=ㄴnode(e.id); if(!n) return false; const tx=n.querySelector('.tx');
741|   const lineH=parseFloat(getComputedStyle(tx).lineHeight)||20;
742|   tx.scrollTop=0; const rr=document.createRange(); rr.selectNodeContents(tx);
743|   const textBottom = tx.textContent.trim() ? rr.getBoundingClientRect().bottom :
    ㄴ, tx.getBoundingClientRect().top;
744|   if(tx.getBoundingClientRect().bottom - textBottom < lineH) return false; // 한 줄 이상 빈자
    ㄴ, 리가 있을 때만
745|   const t=next.els.find(x=>x.type==='text');
746|   const dropNext=(=>{ const after=state.pages[pi+2]; if(after&&after.flowFrom===next.id)
    ㄴ, after.flowFrom=me.id; state.pages.splice(pi+1,1); });
747|   if(!t || !plain(t.html)){ if(next.els.every(x=>x.type==='text'&&!plain(x.html))){ snap();
    ㄴ, dropNext(); renderThumbs(); renderBottom(); } return false; }
748|   snap(); const off=caretOffset(tx);
749|   tx.innerHTML = sanitizeText(tx.innerHTML) + (plain(tx.innerHTML)? ' ':'') + sanitizeText(
    ㄴ, t.html); t.html='';
750|   e.html=tx.innerHTML; if(off!=null) placeCaretAt(tx, off);
751|   const split=flowOverflow(e, true); // 다시 넘치는 만
    ㄴ, 큼은 다음 페이지로 돌려보낸다
752|   if(!split && next.els.every(x=>x.type==='text'&&!plain(x.html))){ dropNext(); toast('다음 페
    ㄴ, 이지 글을 모두 끌어 왔어요'); }
753|   renderThumbs(); renderBottom(); return true;
754| }
755| function placeCaretAt(tx, offset){ const pos=textPositions(tx); const r=document.createRange(
    ㄴ, );
756|   if(!pos.length||offset>=pos.length){ r.selectNodeContents(tx); r.collapse(false); } else {
    ㄴ, const [n,k]=pos[Math.max(0, offset)]; r.setStart(n,k); r.collapse(true); }
757|   const s=getSelection(); s.removeAllRanges(); s.addRange(r); }

```

■ 3. 미디어 배치: 같은 종류의 자리 → 지금 쪽의 빈 공간 → 새 쪽

파일: studio/index.html

```

558|// 견본 사진 자리에 넣기: 자리 안에 비율을 지켜 가운데 맞춤
559|function fillSlot(slotId,kind,src,name,mediaId){
560|  const pi=state.pages.findIndex(p=>p.els.some(e=>e.id===slotId)); if(pi<0) return
  ↵,addMediaPage(kind,src,name,mediaId);
561|  const put=r=>{ const pg=state.pages[pi], i=pg.els.findIndex(e=>e.id===slotId); if(i<0)
  ↵,return; const sl=pg.els[i];
562|    let w=sl.w, h=w*r; if(h>sl.h){ h=sl.h; w=h/r; }
563|    endEdit(); snap(); const m={id:uid(),type:kind,x:sl.x+(sl.w-w)/2,y:sl.y+(sl.h-h)/2,w,h,
  ↵,src,name}; if(mediaId) m.media_id=mediaId;
564|    pg.els.splice(i,1,m); state.cur=pi; state.sel=m.id; renderAll(); toast('사진을 넣었어요');
  ↵,};
565|  if(kind==='image'){ const im=new Image(); im.onload={()=>put(im.naturalHeight/
  ↵,im.naturalWidth||.75); im.onerror={()=>put(.75); im.src=src; }
566|  else put(kind==='video'?9/16:Math.min(.3, 34/CW));
567|}
568|// 사진·영상·노래 넣기: ① 이 페이지에 같은 종류의 빈 자리가 있으면 거기에 ② 이 페이지에 빈 공
  ↵,간이 있으면 거기에 ③ 없을 때만 새 페이지
569|function addMediaPage(kind,src,name,mediaId){
570|  endEdit();
571|  const p0=page(), slot=p0.els.find(e=>e.type==='slot'&&(e.kind||'image')===kind);
572|  if(slot) return fillSlot(slot.id,kind,src,name,mediaId);
573|  snap();
574|  const onThisPage=(w,h)=>{
575|    const filled=p0.els.filter(e=>!(e.type==='text'&&!plain(e.html)));           // 빈 글상자
  ↵,는 빈 공간으로 본다
576|    const bottom=filled.reduce((m,e)=>Math.max(m,e.y+e.h),MT-6), avail=LIM.y1-(bottom+6);
577|    if(avail<Math.min(h,40)) return false;
578|    if(h>avail){ const r=h/w; h=avail; w=kind==='image'?h/r:w; }
579|    const y = filled.length ? bottom+6 : Math.max(MT,(PH-h)/2-14);
580|    const m={id:uid(),type:kind,x:(PW-w)/2,y,w,h,src,name}; if(mediaId) m.media_id=mediaId;
581|    p0.els=p0.els.filter(e=>!(e.type==='text'&&!plain(e.html)&&e.y<y+h&&e.y+e.h>y)); // 겹치는
  ↵,빈 글상자는 치운다
582|    p0.els.push(m); state.sel=m.id; state.editing=false; renderAll();
583|    toast('이 페이지에 담았어요. 「설명 글 넣기」로 설명을 적을 수 있어요'); return true;
584|  };
585|  const finish=(w,h)=>{ if(onThisPage(w,h)) return; // 페이지 안에 가운데 정렬
586|    const m={id:uid(),type:kind,x:(PW-w)/2,y:Math.max(MT,(PH-h)/2-14),w,h,src,name}; if(
  ↵,mediaId) m.media_id=mediaId;
587|    const p={id:uid(),name:`${KIND_KO[kind]} 페이지`,bg:'#FFFFFF',els:[m]};
588|    state.pages.splice(state.cur+1,0,p); state.cur++; state.sel=m.id; state.editing=false;
  ↵,renderAll();
589|    toast('새 페이지에 담았어요. 「설명 글 넣기」로 설명을 적을 수 있어요');
590|  };
591|  if(kind==='image'){ const im=new Image(); im.onload={()=>{ const r=im.naturalHeight/

```

```

    592|   L,im.naturalWidth; let w=CW,h=w*r; if(h>150){ h=150; w=h/r; } finish(w,h); }; im.onerror=(()=>
    593|   L,finish(CW,90); im.src=src; }
    592|   else if(kind==='video'){ finish(CW,CW*9/16); } else finish(CW,34);
    593|}

```

■ 4. 인쇄용 전자책: 영상·노래 자리에 QR 코드 자동 생성

파일: studio/index.html

```

377|// 인쇄용: 영상·노래 자리에 QR 코드(휴대폰으로 찍으면 서가의 그 영상·노래로)
378|const PUBLIC_SHELF = new URL('../shelf/', location.href).href; // 운영 도메인이 정해지면
    L,https://archive4everyone.com/... 로 바꾼다
379|function mediaLink(e){
380|   if(typeof BOOK==='undefined' || !BOOK) return PUBLIC_SHELF;
381|   if(bookRow && !bookRow.is_public && shareToken) return `${PUBLIC_SHELF}#/s/${shareToken}/m/
    L,${e.media_id||''}`; // 비공개 책은 비밀 링크로
382|   return `${PUBLIC_SHELF}#/book/${BOOK}/m/${e.media_id||''}`; }
383|function qrSVG(text){ try{ const q=qrcode(0,'M'); q.addData(text); q.make(); return
    L,q.createSvgTag({cellSize:4,margin:4,scalable:true}); }catch(err){ console.error(err); return
    L,''; } }
384|function qrBlock(e){ const what=e.type==='video'?'영상':'노래';
385|   return `<div class="qr">${qrSVG(mediaLink(e))}<div class="qrt"><b>${what}</b><br>휴대폰 카메
    L,라로 찍으면<br>${what}이 재생돼요</div></div>`; }

```

■ 5. 책 그리기와 안전 검사(서가·내보내기 공용)

파일: lib/book-render.js

```

1|/* 인물대백과사 · 페이지 책 그리기 (서가·내보내기 공용)
2|   저장된 페이지 데이터는 다른 사람이 만든 것일 수 있으므로, 그리기 전에 모든 값을 다시 검사한
    L,다.
3|   - 좌표·크기: 숫자만, 페이지 안으로 제한
4|   - 글: 허용 목록 태그와 글자색/굵기/기울임/밑줄 스타일만
5|   - 미디어: 저장된 주소는 쓰지 않고, media_id로 서버에서 받은 주소만 사용 */
6|(function (global) {
7|   'use strict';
8|   const PW = 152, PH = 225;
9|   const PT_CQW = (4 / 3) / (PW * 3.7795275591) * 100;
10|  const COLOR_RE = /^(#[0-9a-f]{3,8}|rgba?\(\s*\d{1,3}\s*,\s*\d{1,3}\s*,\s*\d{1,3}\s*(,\s*[
    L,\d.]+\s*)?\))$/i;
11|  const HEX6 = /^[0-9a-f]{6}$/i;
12|  const esc = s => String(s ?? '').replace(/([<>"']/g, c => ({ '&': '&amp;', '<': '&lt;'; '>':
    L,'&gt;';, '": '&quot;';, '": '&#39;'; }[c]));
13|  const num = (v, lo, hi, d) => { v = Number(v); return Number.isFinite(v) ? Math.min(hi,
    L,Math.max(lo, v)) : d; };
14|
15|  function sanitizeText(html) {
16|    const d = new DOMParser().parseFromString('<body>' + String(html || '') + '</body>',
    L,'text/html');
17|    (function w(n) {

```

```

18|     [...n.childNodes].forEach(c => {
19|         if (c.nodeType === 3) return;
20|         if (c.nodeType !== 1) { c.remove(); return; }
21|         if (c.tagName === 'FONT') { const sp = d.createElement('span'); const col =
    L,c.getAttribute('color'); if (col && COLOR_RE.test(col)) sp.style.color = col; sp.append(
    L,...c.childNodes); c.replaceWith(sp); c = sp; }
22|         if (!['BR', 'B', 'STRONG', 'I', 'EM', 'U', 'DIV', 'P', 'SPAN'].includes(c.tagName)) {
23|             if (['SCRIPT', 'STYLE', 'IFRAME', 'OBJECT', 'EMBED', 'IMG', 'VIDEO', 'AUDIO', 'SVG',
    L,'MATH'].includes(c.tagName)) { c.remove(); return; }
24|             w(c); c.replaceWith(...c.childNodes); return;
25|         }
26|         const keep = {};
27|         if (c.tagName === 'SPAN') {
28|             const st = c.style;
29|             if (st.color && COLOR_RE.test(st.color.trim())) keep.color = st.color.trim();
30|             if (/^(bold|[6-9]00)$/.test(st.fontWeight)) keep['font-weight'] = '700';
31|             if (st.fontStyle === 'italic') keep['font-style'] = 'italic';
32|             if (/underline/.test(st.textDecoration || st.textDecorationLine || '')) keep['text-
    L,decoration'] = 'underline';
33|         }
34|         [...c.attributes].forEach(a => c.removeAttribute(a.name));
35|         const css = Object.entries(keep).map(([k, v]) => `${k}:${v}`).join(';');
36|         if (css) c.setAttribute('style', css);
37|         w(c);
38|     });
39| })(d.body);
40| return d.body.innerHTML;
41| }
42|
43| const pct = (v, t) => (v / t * 100).toFixed(3) + '%';
44| function box(e) {
45|     const x = num(e.x, 0, PW, 0), y = num(e.y, 0, PH, 0);
46|     const w = num(e.w, 1, PW - x, 10), h = num(e.h, 1, PH - y, 10);
47|     return `left:${pct(x, PW)};top:${pct(y, PH)};width:${pct(w, PW)};height:${pct(h, PH)}`;
48| }
49| function txStyle(e) {
50|     const fs = num(e.fs, 6, 60, 12);
51|     const color = COLOR_RE.test(String(e.color || '')) ? e.color : '#1B2B25';
52|     const align = ['left', 'center', 'right', 'justify'].includes(e.align) ? e.align : 'left';
53|     return `font-size:${(fs * PT_CQW).toFixed(3)}cqw;line-height:${fs >= 20 ? 1.3 : fs >= 15 ?
    L,1.55 : 1.85};text-align:${align};color:${color};font-weight:${e.b ? 700 : 400};font-style:${
    L,e.i ? 'italic' : 'normal'};text-decoration:${e.u ? 'underline' : 'none'}`;
54| }
55|
56| /* urls: { media_id: 서명된 주소 } */
57| function elHTML(e, urls) {

```

```

58|   if (!e || typeof e !== 'object') return '';
59|   let inner = '';
60|   if (e.type === 'text') {
61|     const font = e.font === 'sans' ? 'sans' : 'serif';
62|     inner = `<div class="tx ${font}" style="${txStyle(e)}">${sanitizeText(e.html)}</div>`;
63|   } else if (['image', 'video', 'audio'].includes(e.type)) {
64|     const src = urls && urls[e.media_id];
65|     if (!src) return '';
66|     const s = esc(src);
67|     if (e.type === 'image') inner = ``;
68|     else if (e.type === 'video') inner = `<video src="${s}" controls playsinline preload=
  ↳,"metadata" controlslist="nodownload noplaybackrate noremoteplayback" disablepictureinpicture><
  ↳,/video>`;
69|     else inner = audioPlayer(s); // 내려받기 메뉴가 없는 재생기(BookViewer.wireAudio가 연결)
70|   } else return '';
71|   return `<div class="el" style="${box(e)}">${inner}</div>`;
72| }
73| function audioPlayer(escapedSrc) {
74|   return `<div class="ap" data-src="${escapedSrc}"><button type="button" class="ap-btn"
  ↳,aria-label="재생">▶</button><div class="ap-bar" aria-hidden="true"><i></i></div><span class=
  ↳,"ap-t">0:00</span></div>`;
75| }
76| function pageHTML(p, urls) {
77|   const bg = HEX6.test(String(p.bg || '')) ? p.bg : '#FFFFFF';
78|   const els = Array.isArray(p.els) ? p.els : [];
79|   return `<div class="bk-pg" style="background:${bg}">${els.map(e => elHTML(e, urls)).join(
  ↳,','></div>`;
80| }
81| const CSS = `.bk-pg{position:relative;aspect-ratio:152/225;container-type:inline-size;
  ↳,overflow:hidden;background:#fff}
82|.bk-pg .el{position:absolute}
83|.bk-pg .tx{width:100%;height:100%;overflow:hidden;word-break:keep-all;overflow-wrap:anywhere;
  ↳,white-space:pre-wrap;font-family:'Noto Serif KR','Batang',serif}
84|.bk-pg .tx.sans{font-family:'Pretendard',-apple-system,'Apple SD Gothic Neo','Malgun Gothic',
  ↳,sans-serif}
85|.bk-pg img,.bk-pg video{width:100%;height:100%;object-fit:cover;display:block;border-radius:
  ↳,.6cqW}
86|.bk-pg .audio{width:100%;height:100%;border:.3cqW solid #E6EBE7;border-radius:1.6cqW;display:
  ↳,flex;flex-direction:column;justify-content:center;align-items:center;gap:1.4cqW;background:
  ↳,#F6FAF8}
87|.bk-pg .audio .lb{font:2.8cqW sans-serif;color:#4A5A53}.bk-pg .audio audio{width:92%}`;
88|
89| global.BookRender = { PW, PH, sanitizeText, pageHTML, CSS, esc, audioPlayer };
90|})(window);

```

■ 6. 책 보기: 두 쪽 펼침 · 3D 책장 넘김 · 노래 재생기 · 복사 방지

파일: lib/book-viewer.js

```

1| /* 인물대백과사 · 책 보기 부품 (서가·비밀 링크·화면용 전자책 공용)
2|   1) 책처럼 넘기는 보기: 넓은 화면은 두 쪽 펼침(1·2쪽 → 3·4쪽), 좁은 화면은 한 쪽. 책장이 넘
   ↳어가는 효과, 쪽 번호,
3|     < > 버튼 · ↳ → 키·마우스 휠(스크롤)·손가락으로 밀기·책 가장자리 누르기
4|   2) 진행 막대가 잘 보이는 노래 재생기(내려받기 메뉴 없음)
5|   3) 저작권 보호: 우클릭·끌기·복사·인쇄 막기, 영상 내려받기 메뉴 숨기기
6|   ※ 화면 녹화·개발자 도구까지 막을 수는 없다. '가볍게 퍼 가기'를 막는 장치다. */
7|(function (global) {
   : (중략)
30|.bv-hot: hover{background: linear-gradient(to right, rgba(11, 51, 36, .06), transparent)}.bv-hot.r:
   ↳hover{background: linear-gradient(to left, rgba(11, 51, 36, .06), transparent)}
31|.bv-nav{display: flex; align-items: center; justify-content: center; gap: 14px; margin: 14px 0 2px}
32|.bv-nav button{width: 46px; height: 46px; border-radius: 50%; border: 1.5px solid #CFC8AE; background:
   ↳, #fff; color: #0B3324; font-size: 22px; cursor: pointer; display: grid; place-items: center}
33|.bv-nav button: hover: not(: disabled){background: #0B3324; color: #fff}.bv-nav button: disabled{
   ↳, opacity: .3; cursor: default}
34|.bv-no{min-width: 96px; text-align: center; font-weight: 700; color: #0B3324; font-size: 15px; font-
   ↳variant-numeric: tabular-nums}
35|.bv-hint{text-align: center; font-size: 12.5px; color: #9AA7A1; margin: 2px 0 0}
36| @media (prefers-reduced-motion: reduce){.bv-leaf, .bv-leaf>div::after{transition: none}}
37|.ap{width: 100%; height: 100%; display: flex; align-items: center; gap: 3cqw; padding: 0 3.5cqw; border:
   ↳, 3cqw solid #D8E6DF; border-radius: 1.6cqw; background: linear-gradient(135deg, #F1F7F4, #fff); font-
   ↳family: 'Pretendard', sans-serif}
38|.ap-btn{flex: none; width: 10cqw; height: 10cqw; border-radius: 50%; border: none; background: #0F5B48;
   ↳, color: #fff; font-size: 4cqw; cursor: pointer; display: grid; place-items: center; padding: 0}
39|.ap-btn: hover{background: #0B4536}
40|.ap-bar{flex: 1; height: 2.2cqw; border-radius: 2cqw; background: #DCE8E2; position: relative; cursor:
   ↳, pointer; overflow: hidden}
41|.ap-bar i{position: absolute; left: 0; top: 0; bottom: 0; width: 0; background: linear-gradient(90deg,
   ↳, #E0A52E, #C0392B); border-radius: 2cqw}
42|.ap.playing .ap-bar{box-shadow: 0 0 0 .4cqw rgba(224, 165, 46, .25)}
43|.ap-t{flex: none; font-size: 2.8cqw; font-weight: 700; color: #0B4536; min-width: 14cqw; text-align:
   ↳, right; font-variant-numeric: tabular-nums}
44|.bv-protect, .bv-protect *{-webkit-user-select: none; user-select: none; -webkit-touch-callout:
   ↳, none}
45|.bv-protect img{pointer-events: none; -webkit-user-drag: none}
46| `;
47|   const fmt = s => { s = Math.max(0, Math.floor(s || 0)); return Math.floor(s / 60) + ':' +
   ↳, String(s % 60).padStart(2, '0'); };
48|
49|   /* 노래 재생기: data-src 가 있는 .ap 를 찾아 연결한다(한 번에 한 곡만 재생) */
50|   function wireAudio(root) {
51|     root.querySelectorAll('.ap[data-src]').forEach(ap => {
52|       if (ap._wired) return; ap._wired = true;

```

```

53|     const a = new Audio(); a.preload = 'none'; a.src = ap.dataset.src; ap.removeAttribute(
    ↓,'data-src');
54|     a.setAttribute('controlsList', 'nodownload');
55|     const btn = ap.querySelector('.ap-btn'), bar = ap.querySelector('.ap-bar'), fill =
    ↓,ap.querySelector('.ap-bar i'), t = ap.querySelector('.ap-t');
56|     const upd = () => { const d = a.duration || 0; fill.style.width = d ? (a.currentTime /
    ↓,d * 100) + '%' : '0'; t.textContent = fmt(a.currentTime) + (d ? ' / ' + fmt(d) : ''); };
57|     btn.onclick = () => { if (a.paused) { document.querySelectorAll('audio').forEach(x => x
    ↓,!= a && x.pause()); (global.__bvAudios || []).forEach(x => x != a && x.pause()); a.play();
    ↓,} else a.pause(); };
58|     a.addEventListener('play', () => { btn.textContent = '▶'; btn.setAttribute('aria-
    ↓,label', '멈춤'); ap.classList.add('playing'); });
59|     a.addEventListener('pause', () => { btn.textContent = '▶'; btn.setAttribute('aria-
    ↓,label', '재생'); ap.classList.remove('playing'); });
60|     a.addEventListener('timeupdate', upd); a.addEventListener('loadedmetadata', upd);
    ↓,a.addEventListener('ended', () => { a.currentTime = 0; upd(); });
61|     bar.onclick = ev => { const r = bar.getBoundingClientRect(); if (a.duration)
    ↓,a.currentTime = (ev.clientX - r.left) / r.width * a.duration; };
62|     a._ap = ap; (global.__bvAudios = global.__bvAudios || []).push(a); ap._audio = a;
63|   });
64| }
65| /* 영상: 내려받기·PIP 메뉴 숨기기 */
66| function wireVideo(root) {
67|   root.querySelectorAll('video').forEach(v => { v.setAttribute('controlsList', 'nodownload
    ↓,noplaybackrate noremoteplayback'); v.disablePictureInPicture = true; v.setAttribute(
    ↓,'disablePictureInPicture', ''); });
68| }
69| /* 저작권 보호: 우클릭·끌기·복사·잘라내기·인쇄 */
70| function protect(root, opts = {}) {
71|   root.classList.add('bv-protect');
72|   const stop = e => { e.preventDefault(); if (opts.onBlocked) opts.onBlocked(e.type); };
73|   ['contextmenu', 'dragstart', 'copy', 'cut', 'selectstart'].forEach(t =>
    ↓,root.addEventListener(t, stop));
74|   if (opts.blockPrint !== false && !document.getElementById('bv-noprint')) {
75|     const st = document.createElement('style'); st.id = 'bv-noprint';
76|     st.textContent = '@media print{body>*{display:none!important}body::before{content:"이 책
    ↓,은 작가의 저작권 보호를 위해 인쇄할 수 없어요.";display:block;font:16px sans-serif;padding:
    ↓,40px;text-align:center}}';
77|     document.head.appendChild(st);
78|     document.addEventListener('keydown', e => { if ((e.ctrlKey || e.metaKey) && ['p',
    ↓,'s'].includes(e.key.toLowerCase())) { e.preventDefault(); if (opts.onBlocked) opts.onBlocked(
    ↓,e.key.toLowerCase() === 'p' ? 'print' : 'save'); } });
79|   }
80| }
81| /* 책 보기: pagesHTML(쪽마다 HTML 문자열 배열)을 container 에 그린다.
82|   넓은 화면(두 쪽이 들어가면) = 펼침면 [1·2] → [3·4] ..., 좁은 화면 = 한 쪽씩.

```

```

83|     보이는 쪽만 그리고(무거운 영상·노래를 한꺼번에 불러오지 않음), 넘길 때 책장이 넘어가는 효
    ㄴ,과를 준다. */
84|     function flip(container, pagesHTML, opts = {}) {
85|         const n = pagesHTML.length, pre = opts.idPrefix || 'c';
86|         const reduce = global.matchMedia && global.matchMedia('(prefers-reduced-motion:
    ㄴ,reduce)').matches;
87|         container.innerHTML = `<div class="bv"><div class="bv-stage"><div class="bv-book"
    ㄴ,tabindex="0" aria-label="책 (← → 키, 스크롤, 밀기로 넘기기)"></div></div>
88|             <div class="bv-nav"><button type="button" class="bv-prev" aria-label="앞 쪽"></button><
    ㄴ,span class="bv-no" aria-live="polite"></span><button type="button" class="bv-next" aria-label=
    ㄴ,"다음 쪽"></button></div>
89|             <p class="bv-hint">< > · ← → 키 · 스크롤 · 밀어서 넘겨요</p></div>`;
90|         const stage = container.querySelector('.bv-stage'), book = container.querySelector('.bv-
    ㄴ,book'), no = container.querySelector('.bv-no'), prev = container.querySelector('.bv-prev'),
    ㄴ,next = container.querySelector('.bv-next');
91|         let two = false, cur = 0, busy = false;                // cur: 보이는 첫 쪽(펼침면이면 짝
    ㄴ,수)
92|         const step = () => two ? 2 : 1;
93|         const startOf = i => two ? i - (i % 2) : i;
94|         function pageEl(i) {
95|             const d = document.createElement('div');
96|             if (i < 0 || i >= n) { d.className = 'bv-blank'; return d; }
97|             d.className = 'bv-page'; d.id = pre + i; d.setAttribute('aria-label', (i + 1) + '쪽');
98|             d.innerHTML = pagesHTML[i] + `<div class="bv-num">${i + 1}</div>`;
99|             wireAudio(d); wireVideo(d); if (opts.onPage) opts.onPage(d, i);
100|             return d;
101|         }
102|         const slot = side => { const s = document.createElement('div'); s.className = 'bv-slot ' +
    ㄴ,side; return s; };
103|         function layout() {
104|             const W = Math.max(240, (container.clientWidth || global.innerWidth) - 8);
105|             const H = Math.max(300, global.innerHeight - (opts.reserve ?? 200));
106|             const t = opts.spread !== false && W >= 700 && n > 1;
107|             const pw = t ? Math.min(W / 2, H * 152 / 225, 470) : Math.min(W, H * 152 / 225, 540);
108|             book.style.setProperty('--pw', Math.floor(pw) + 'px');
109|             if (t !== two) { two = t; cur = startOf(cur); render(); }
110|         }
111|         function render() {
112|             book.classList.toggle('two', two); book.innerHTML = '';
113|             if (two) { const l = slot('l'), r = slot('r'); l.append(pageEl(cur)); r.append(pageEl(
    ㄴ,cur + 1)); book.append(l, r); }
114|             else { const one = slot('one'); one.append(pageEl(cur)); book.append(one); }
115|             const hl = document.createElement('div'), hr = document.createElement('div');
116|             hl.className = 'bv-hot l'; hl.title = '앞 쪽'; hl.onclick = () => go(cur - step());
117|             hr.className = 'bv-hot r'; hr.title = '다음 쪽'; hr.onclick = () => go(cur + step());
118|             book.append(hl, hr); status();

```

```

119|     }
120|     function status() {
121|         const last = two ? Math.min(cur + 2, n) : cur + 1;
122|         no.textContent = (last - cur > 1 ? `${cur + 1}-${last}` : `${cur + 1}`) + ` / ${n}`;
123|         prev.disabled = cur <= 0; next.disabled = cur + step() >= n;
124|         (global.__bvAudios || []).forEach(a => { if (a._ap && !a._ap.isConnected) a.pause(); });
    ↵, // 넘긴 쪽의 노래는 멈춤
125|         global.__bvAudios = (global.__bvAudios || []).filter(a => !a._ap || a._ap.isConnected);
126|     }
127|     // 책장 넘기기: 넘어가는 한 장(leaf)의 앞면·뒷면에 쪽을 붙여 3D로 돌린다
128|     function turn(dir) {
129|         const slots = book.querySelectorAll('.bv-slot');
130|         const leaf = document.createElement('div'); leaf.className = 'bv-leaf ' + (dir > 0 ?
    ↵, 'nx' : 'pv');
131|         const f = document.createElement('div'), b = document.createElement('div'); f.className
    ↵, = 'f'; b.className = 'b'; leaf.append(f, b);
132|         const pw = parseFloat(book.style.getPropertyValue('--pw'));
133|         let done;
134|         if (two && dir > 0) {                                // 오른쪽 장이 왼쪽으로
135|             const [L, R] = slots; const nl = pageEl(cur + 2), nr = pageEl(cur + 3);
136|             f.append(R.firstElementChild); b.append(nl); R.replaceChildren(nr);
137|             leaf.style.left = pw + 'px'; leaf.style.transformOrigin = 'left center';
138|             done = () => L.replaceChildren(nl);
139|             requestAnimationFrame(() => requestAnimationFrame(() => { leaf.classList.add('go');
    ↵, leaf.style.transform = 'rotateY(-180deg)'; }));
140|         } else if (two) {                                    // 왼쪽 장이 오른쪽으로
141|             const [L, R] = slots; const nl = pageEl(cur - 2), nr = pageEl(cur - 1);
142|             f.append(L.firstElementChild); b.append(nr); L.replaceChildren(nl);
143|             leaf.style.left = '0px'; leaf.style.transformOrigin = 'right center';
144|             done = () => R.replaceChildren(nr);
145|             requestAnimationFrame(() => requestAnimationFrame(() => { leaf.classList.add('go');
    ↵, leaf.style.transform = 'rotateY(180deg)'; }));
146|         } else if (dir > 0) {                                // 한 쪽 보기: 지금 쪽이 왼쪽으로 넘어감
147|             const [S] = slots; const np = pageEl(cur + 1);
148|             f.append(S.firstElementChild); b.append(pageEl(-1)); S.replaceChildren(np);
149|             leaf.style.left = '0px'; leaf.style.transformOrigin = 'left center';
150|             done = () => {};
151|             requestAnimationFrame(() => requestAnimationFrame(() => { leaf.classList.add('go');
    ↵, leaf.style.transform = 'rotateY(-180deg)'; }));
152|         } else {                                            // 한 쪽 보기: 앞 쪽이 왼쪽에서 덮어옴
153|             const [S] = slots; const pp = pageEl(cur - 1);
154|             f.append(pp); b.append(pageEl(-1));
155|             leaf.style.left = '0px'; leaf.style.transformOrigin = 'left center';
    ↵, leaf.style.transition = 'none'; leaf.style.transform = 'rotateY(-180deg)';
156|             done = () => S.replaceChildren(pp);
157|             requestAnimationFrame(() => requestAnimationFrame(() => { leaf.style.transition = '';

```

```

    L,leaf.classList.add('go'); leaf.style.transform = 'rotateY(0deg)'; }));
158|     }
159|     book.append(leaf); busy = true;
160|     let ended = false;
161|     const end = () => { if (ended) return; ended = true; done(); leaf.remove(); cur += dir *
    L,step(); status(); busy = false; };
162|     leaf.addEventListener('transitionend', e => { if (e.target === leaf && e.propertyName ==
    L,= 'transform') end(); });
163|     setTimeout(end, 1000); // 효과가 꺼진 환경에서도 반드시 끝나게
164| }
165| function go(i, smooth = true) {
166|     if (busy) return;
167|     i = startOf(Math.max(0, Math.min(n - 1, i)));
168|     if (i === cur) return;
169|     if (smooth && !reduce && Math.abs(i - cur) === step()) turn(i > cur ? 1 : -1);
170|     else { cur = i; render(); }
171| }
172| prev.onclick = () => go(cur - step()); next.onclick = () => go(cur + step());
173| book.addEventListener('keydown', e => { if (e.key === 'ArrowRight' || e.key ===
    L,'PageDown') { e.preventDefault(); go(cur + step()); } else if (e.key === 'ArrowLeft' || e.key
    L,=== 'PageUp') { e.preventDefault(); go(cur - step()); } });
174| document.addEventListener('keydown', e => { if (!container.isConnected ||
    L,e.target.closest && e.target.closest('.bv-book')) return; if (/^(INPUT|TEXTAREA|SELECT)$/
    L,.test(document.activeElement.tagName) || document.activeElement.isContentEditable) return; if
    L,(e.key === 'ArrowRight') go(cur + step()); else if (e.key === 'ArrowLeft') go(cur - step());
    L,});
175| // 마우스 휠(스크롤)로 넘기기: 한 번 굴리면 한 번 넘김
176| let wheelLock = 0, acc = 0;
177| stage.addEventListener('wheel', e => {
178|     const d = Math.abs(e.deltaX) > Math.abs(e.deltaY) ? e.deltaX : e.deltaY;
179|     if ((d > 0 && next.disabled) || (d < 0 && prev.disabled)) return; // 끝 쪽에서는 화면
    L,스크롤을 막지 않음
180|     e.preventDefault();
181|     if (busy || Date.now() < wheelLock) return;
182|     acc += d; if (Math.abs(acc) < 30) return;
183|     go(cur + (acc > 0 ? step() : -step())); acc = 0; wheelLock = Date.now() + 700;
184| }, { passive: false });
185| // 손가락으로 밀기
186| let tx = null, ty = null;
187| stage.addEventListener('touchstart', e => { const t = e.changedTouches[0]; tx = t.clientX;
    L,ty = t.clientY; }, { passive: true });
188| stage.addEventListener('touchend', e => { if (tx === null) return; const t =
    L,e.changedTouches[0], dx = t.clientX - tx, dy = t.clientY - ty; tx = null;
189|     if (Math.abs(dx) > 40 && Math.abs(dx) > Math.abs(dy)) go(cur + (dx < 0 ? step() : -step(
    L,))); }, { passive: true });
190| addEventListener('resize', () => { if (container.isConnected) layout(); });

```

```

191|   layout(); if (!book.firstChild) render();
192|   const api = { go: (i, smooth = true) => go(i, smooth), get current() { return cur; }, get
    ↳,spread() { return two; }, count: n };
193|   container._bv = api;
194|   return api;
195| }
196| function injectCSS() { if (document.getElementById('bv-css')) return; const st =
    ↳,document.createElement('style'); st.id = 'bv-css'; st.textContent = CSS;
    ↳,document.head.appendChild(st); }
197| global.BookViewer = { CSS, flip, wireAudio, wireVideo, protect, injectCSS };
198|})(window);

```

■ 7. 서버: 비밀 링크로 책 보기

파일: supabase/functions/shared-book/index.ts

```

1|// 비밀 링크로 책 보기: 링크 열쇠(token)가 맞으면 그 책의 '발행본'과 미디어 임시 주소(1시간)를
    ↳,돌려준다.
2|// - 로그인 없이 부르는 함수라 verify_jwt=false. 대신 추측할 수 없는 긴 열쇠(32자 이상)로만 연
    ↳,다.
3|// - 서비스 키는 이 함수 안에서만 쓰이고 브라우저로 나가지 않는다.
4|// - 휴지통에 있거나 관리자가 숨긴 책은 열리지 않는다. 작가가 링크를 끊으면(열쇠 삭제) 바로 막
    ↳,한다.
5|import { createClient } from "npm:@supabase/supabase-js@2";
6|
7|const CORS = {
8|  "Access-Control-Allow-Origin": "*",
9|  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type",
10|  "Access-Control-Allow-Methods": "POST, OPTIONS",
11|};
12|const json = (body: unknown, status = 200) =>
13|  new Response(JSON.stringify(body), { status, headers: { ...CORS, "Content-Type":
    ↳,"application/json" } });
14|
15|Deno.serve(async (req: Request) => {
16|  if (req.method === "OPTIONS") return new Response("ok", { headers: CORS });
17|  if (req.method !== "POST") return json({ error: "method" }, 405);
18|  let token = "";
19|  try { token = String((await req.json())?.token ?? ""); } catch { /* 빈 요청 */ }
20|  if (!/^[A-Za-z0-9_]{32,64}$/.test(token)) return json({ error: "not_found" }, 404);
21|
22|  const sb = createClient(Deno.env.get("SUPABASE_URL")!, Deno.env.get(
    ↳,"SUPABASE_SERVICE_ROLE_KEY")!, {
23|    auth: { persistSession: false },
24|  });
25|  const { data: share } = await sb.from("book_shares").select("book_id").eq("token",
    ↳,token).maybeSingle();
26|  if (!share) return json({ error: "not_found" }, 404);

```

```
27|  const { data: book } = await sb.from("books").select("id,deleted_at,admin_hidden").eq("id",
    |  ↳share.book_id).maybeSingle();
28|  if (!book || book.deleted_at || book.admin_hidden) return json({ error: "not_found" }, 404);
29|  const { data: snap } = await sb.from("book_snapshots").select("title,author_name,pages").eq(
    |  ↳"book_id", book.id).maybeSingle();
30|  if (!snap) return json({ error: "not_found" }, 404);
31|
32|  // 발행본에 쓰인 미디어만 서명한다(이 책의 파일만)
33|  const ids = new Set<string>();
34|  for (const p of Array.isArray(snap.pages) ? snap.pages : []) {
35|    for (const e of Array.isArray(p?.els) ? p.els : []) if (e && typeof e.media_id ===
        |  ↳"string") ids.add(e.media_id);
36|  }
37|  const urls: Record<string, string> = {};
38|  if (ids.size) {
39|    const { data: ms } = await sb.from("media").select("id,storage_path").eq("book_id",
        |  ↳book.id).in("id", [...ids]);
40|    const list = (ms ?? []).filter((m) => m.storage_path);
41|    if (list.length) {
42|      const { data: signed } = await sb.storage.from("book-media").createSignedUrls(list.map((
        |  ↳m) => m.storage_path), 3600);
43|      (signed ?? []).forEach((s, i) => { if (s?.signedUrl) urls[list[i].id] = s.signedUrl; });
44|    }
45|  }
46|  return json({ title: snap.title, author_name: snap.author_name, pages: snap.pages, urls });
47|});
```



```
40|     let offset = 0;
41|     for (;;) {
42|         const { data: files } = await admin.storage.from(BUCKET).list(`${uid}/${f.name}`, {
43|             limit: 1000, offset });
44|         if (!files?.length) break;
45|         files.forEach((x) => x.id && paths.add(`${uid}/${f.name}/${x.name}`));
46|         if (files.length < 1000) break;
47|         offset += 1000;
48|     }
49|     const list = [...paths].filter((p) => p.startsWith(uid + "/")); // 내 폴더 밖은 절대 건드리
50|     지 않음
51|     for (let i = 0; i < list.length; i += 100) {
52|         const { error } = await admin.storage.from(BUCKET).remove(list.slice(i, i + 100));
53|         if (error) return json({ error: "storage_failed" }, 500);
54|     }
55|     // ② 계정 삭제(연결된 DB 기록은 cascade로 함께 삭제)
56|     const { error: delErr } = await admin.auth.admin.deleteUser(uid);
57|     if (delErr) return json({ error: "delete_failed" }, 500);
58|     return json({ ok: true, files: list.length });
59| });
```

■ 9. 데이터 구조와 접근 규칙

파일: supabase/migrations/20260930_0001_core_schema.sql

```

1|-- 인물대백과사 · 멀티 전자책 에디터 - 핵심 스키마 v1
2|-- 원칙: (1) 내 책은 언제든지 통째로 내려받을 수 있다(내보내기 우선)
3|--          (2) 표준 Postgres + S3 호환 저장소만 사용(이사 가능)
4|--          (3) "영구 보존" 문구를 약속하지 않는다
5|--          (4) 본인 책은 본인만, 공개한 책만 서가에서 보인다(RLS)
6|
7| : (중략)
8|
13|-- _____ 프로필 _____
14|create table public.profiles (
15|  id uuid primary key references auth.users(id) on delete cascade,
16|  display_name text not null default '' check (char_length(display_name) <= 40),
17|  created_at timestamptz not null default now(),
18|  updated_at timestamptz not null default now()
19|);
20| : (중략)
21|
45|-- _____ 책 _____
46|create table public.books (
47|  id uuid primary key default gen_random_uuid(),
48|  owner_id uuid not null references auth.users(id) on delete cascade,
49|  title text not null default '제목 없는 책' check (char_length(title) between 1 and 120),
50|  subtitle text not null default '' check (char_length(subtitle) <= 200),
51|  author_name text not null default '' check (char_length(author_name) <= 40),
52|  cover_media_id uuid,
53|  is_public boolean not null default false, -- 작가가 서가 공개를 누르면 true
54|  published_at timestamptz,
55|  admin_hidden boolean not null default false, -- 관리자만 변경(함수 admin_set_hidden)
56|  admin_note text not null default '',
57|  deleted_at timestamptz, -- 소프트 삭제(복구 가능)
58|  created_at timestamptz not null default now(),
59|  updated_at timestamptz not null default now()
60|);
61|create index books_owner_idx on public.books(owner_id) where deleted_at is null;
62|create index books_shelf_idx on public.books(published_at desc)
63|  where is_public and not admin_hidden and deleted_at is null;
64|create trigger books_updated before update on public.books
65|  for each row execute function public.set_updated_at();
66| : (중략)
67|
81|-- _____ 미디어(이미지/영상/노래) _____
82|create table public.media (
83|  id uuid primary key default gen_random_uuid(),
84|  book_id uuid not null references public.books(id) on delete cascade,
85|  owner_id uuid not null references auth.users(id) on delete cascade,
86|  kind text not null check (kind in ('image','video','audio','link')),
87|  storage_path text, -- 저장소 파일 경로: {owner_id}/{book_id}/{파일명}

```

```

88| external_url text check (external_url is null or external_url ~* '^https://'),
89| caption text not null default '' check (char_length(caption) <= 300),
90| size_bytes bigint not null default 0 check (size_bytes >= 0),
91| created_at timestamptz not null default now(),
92| check (storage_path is not null or external_url is not null)
93|);
94|create index media_book_idx on public.media(book_id);
95|alter table public.books
96| add constraint books_cover_fk foreign key (cover_media_id) references public.media(id) on
  |delete set null;
97|
98|-- ————— 신고 —————
99|create table public.reports (
100| id uuid primary key default gen_random_uuid(),
101| book_id uuid not null references public.books(id) on delete cascade,
102| reporter_id uuid not null references auth.users(id) on delete cascade,
103| reason text not null check (char_length(reason) between 1 and 1000),
104| status text not null default 'open' check (status in ('open','resolved','dismissed')),
105| created_at timestamptz not null default now(),
106| unique (book_id, reporter_id)
107|);
  | : (중략)
109|-- ————— 공개 여부 판별 —————
110|create or replace function public.book_is_public(b uuid)
111|returns boolean language sql stable security definer
112|set search_path = ''
113|as $$ select exists (select 1 from public.books
114| where id = b and is_public and not admin_hidden and deleted_at is null) $$;
115|
116|create or replace function public.book_is_mine(b uuid)
117|returns boolean language sql stable security definer
118|set search_path = ''
119|as $$ select exists (select 1 from public.books
120| where id = b and owner_id = (select auth.uid())) $$;
  | : (중략)
142|-- ————— RLS —————
143|alter table public.profiles enable row level security;
144|alter table public.admin_users enable row level security;
145|alter table public.books enable row level security;
146|alter table public.chapters enable row level security;
147|alter table public.media enable row level security;
148|alter table public.reports enable row level security;
149|
150|-- profiles
151|create policy profiles_select on public.profiles for select to authenticated
152| using (id = (select auth.uid()) or public.is_admin());

```

```
153|create policy profiles_update on public.profiles for update to authenticated
154|  using (id = (select auth.uid())) with check (id = (select auth.uid()));
155|
156|-- books: 본인 전체, 관리자 조회. 공개 열람은 뷰(shelf_books)와 본문 정책으로.
157|create policy books_select_own on public.books for select to authenticated
158|  using (owner_id = (select auth.uid()) or public.is_admin());
159|create policy books_insert_own on public.books for insert to authenticated
160|  with check (owner_id = (select auth.uid()));
161|create policy books_update_own on public.books for update to authenticated
162|  using (owner_id = (select auth.uid())) with check (owner_id = (select auth.uid()));
163|create policy books_delete_own on public.books for delete to authenticated
164|  using (owner_id = (select auth.uid()));
165|-- 작가는 관리자 전용 컬럼을 바꿀 수 없다
166|revoke update on public.books from authenticated;
167|grant update (title, subtitle, author_name, cover_media_id, is_public, published_at,
    _deleted_at)
168|  on public.books to authenticated;
169|
    : (중략)
176|-- media
177|create policy media_select on public.media for select to anon, authenticated
178|  using (public.book_is_mine(book_id) or public.book_is_public(book_id) or public.is_admin());
179|create policy media_write on public.media for all to authenticated
180|  using (owner_id = (select auth.uid()) and public.book_is_mine(book_id))
181|  with check (owner_id = (select auth.uid()) and public.book_is_mine(book_id));
    : (중략)
191|-- ----- 파일 저장소(비공개 버킷) -----
192|insert into storage.buckets (id, name, public, file_size_limit)
193|values ('book-media', 'book-media', false, 52428800) -- 파일당 50MB
194|on conflict (id) do nothing;
195|
196|create policy "book-media owner all" on storage.objects for all to authenticated
197|  using (bucket_id = 'book-media' and (storage.foldername(name))[1] = (select auth.uid()::
    _text)
198|  with check (bucket_id = 'book-media' and (storage.foldername(name))[1] = (select auth.uid(
    _))::text);
199|create policy "book-media public read" on storage.objects for select to anon, authenticated
200|  using (bucket_id = 'book-media' and exists (
201|    select 1 from public.media m
202|    where m.storage_path = storage.objects.name and public.book_is_public(m.book_id));
203|create policy "book-media admin read" on storage.objects for select to authenticated
204|  using (bucket_id = 'book-media' and public.is_admin());
```

■ 10. 쪽 단위 저장 표(편집기의 쪽을 그대로 저장)

파일: supabase/migrations/20260930_0005_pages.sql

```

1|-- 새 편집기(스튜디오)용 페이지 저장 구조.
2|-- 책은 format='chapters'(옛 에디터) 또는 'pages'(스튜디오). 옛 chapters 내용은 옮긴 뒤에도 지
   |,우지 않고 보관한다.
3|alter table public.books add column format text not null default 'chapters'
4| check (format in ('chapters','pages'));
5|grant update (format) on public.books to authenticated;
6|
7|create table public.pages (
8|  id uuid primary key,
9|  book_id uuid not null references public.books(id) on delete cascade,
10| position integer not null default 0,
11| name text not null default '' check (char_length(name) <= 60),
12| bg text not null default '#FFFFFF' check (bg ~* '^#[0-9a-f]{6}$'),
13| flow_from uuid,          -- 자동 넘김으로 이어진 앞 페이지
14| els jsonb not null default '[]'::jsonb
15|   check (jsonb_typeof(els) = 'array' and pg_column_size(els) < 400000),
16| created_at timestamptz not null default now(),
17| updated_at timestamptz not null default now()
18|);
19|create index pages_book_idx on public.pages(book_id, position);
20|create trigger pages_updated before update on public.pages
21| for each row execute function public.set_updated_at();
22|
23|alter table public.pages enable row level security;
24|create policy pages_select on public.pages for select to anon, authenticated
25| using (private.book_is_mine(book_id) or private.book_is_public(book_id) or private.is_admin(
   |,));
26|create policy pages_write on public.pages for all to authenticated
27| using (private.book_is_mine(book_id)) with check (private.book_is_mine(book_id));
28|
29|-- 서가 뷰에 format 추가
30|drop view public.shelf_books;
31|create view public.shelf_books with (security_invoker = true) as
32| select id, title, subtitle, author_name, cover_media_id, published_at, format
33|   from public.books
34|  where is_public and not admin_hidden and deleted_at is null;
35|revoke all on public.shelf_books from public;
36|grant select on public.shelf_books to anon, authenticated;

```